BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 7

Roll no: 51 PRN No: 12411033 Name of Student: Manjiri Kulkarni

Subject Name: Deep Learning

Problem Statement:

Implement transfer learning using pre-trained AlexNet, VGG-16, ResNet50, EfficientNetB0 model for image classification and compare their performance.

Algorithm:

1. Start the program.
2. Import the required libraries and load the flower dataset.
3. Resize images to 224 × 224 pixels and split the dataset into training and validation sets.
4. Build the AlexNet model and load pretrained VGG16, ResNet50, and EfficientNetB0 models.
5. Add classification layers to each model and compile them using the Adam optimizer and categorical cross-entropy loss.
6. Train all four models for 5 epochs.
7. Evaluate each model using validation loss and accuracy.
8. Compare the performance of all four models using a results table and accuracy graph.
9. Plot validation accuracy and training versus validation accuracy across epochs.
10. Identify the best-performing model based on validation accuracy.
11. Display the best model's accuracy and loss.
12. Stop the program.

Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG16, ResNet50, EfficientNetB0
from tensorflow.keras.applications.vgg16 import preprocess_input as vgg_preprocess
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
print("TensorFlow version:", tf.__version__)
print("Available GPU:", tf.config.list_physical_devices("GPU"))
dataset_path = tf.keras.utils.get_file(
    "flower_photos",

origin="https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz",
    untar=True
)
print("Downloaded dataset path:", dataset_path)
print("Folders:", os.listdir(dataset_path))
nested_path = os.path.join(dataset_path, "flower_photos")
if os.path.isdir(nested_path):
    dataset_path = nested_path
print("Final dataset path:", dataset_path)
print("Contents:", os.listdir(dataset_path))
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
SEED = 123
EPOCHS = 5
train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="int"
)
class_names = train_ds.class_names
num_classes = len(class_names)
print("Class names:", class_names)
print("Number of classes:", num_classes)
assert num_classes == 5, "Dataset should contain five flower classes!"
```

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
alexnet_model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(96, kernel_size=11, strides=4, activation="relu"),
    layers.MaxPooling2D(pool_size=3, strides=2),
    layers.Conv2D(256, kernel_size=5, padding="same", activation="relu"),
    layers.MaxPooling2D(pool_size=3, strides=2),
    layers.Conv2D(384, kernel_size=3, padding="same", activation="relu"),
    layers.Conv2D(384, kernel_size=3, padding="same", activation="relu"),
    layers.Conv2D(256, kernel_size=3, padding="same", activation="relu"),
    layers.MaxPooling2D(pool_size=3, strides=2),
    layers.GlobalAveragePooling2D(),
    layers.Dense(512, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation="softmax")
])
alexnet_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
alexnet_model.summary()
alexnet_history = alexnet_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
alexnet_loss, alexnet_accuracy = alexnet_model.evaluate(val_ds)
print("AlexNet Validation Loss:", alexnet_loss)
print("AlexNet Validation Accuracy:", alexnet_accuracy)
print("AlexNet Validation Accuracy (%):", round(alexnet_accuracy * 100, 2))
vgg_base = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
vgg_base.trainable = False
vgg_model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Lambda(vgg_preprocess),
    vgg_base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation="softmax")
])
```

```
vgg_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("VGG16 output shape:", vgg_model.output_shape)
vgg_history = vgg_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)

vgg_loss, vgg_accuracy = vgg_model.evaluate(val_ds)
print("VGG16 Validation Loss:", vgg_loss)
print("VGG16 Validation Accuracy:", vgg_accuracy)
print("VGG16 Validation Accuracy (%):", round(vgg_accuracy * 100, 2))
resnet_base = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
resnet_base.trainable = False
resnet_model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Lambda(resnet_preprocess),
    resnet_base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes,  activation="softmax")
])
resnet_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("ResNet50 output shape:", resnet_model.output_shape)
resnet_history = resnet_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
resnet_loss, resnet_accuracy = resnet_model.evaluate(val_ds)
print("ResNet50 Validation Loss:", resnet_loss)
print("ResNet50 Validation Accuracy:", resnet_accuracy)
print("ResNet50 Validation Accuracy (%):", round(resnet_accuracy * 100, 2))
efficientnet_base = EfficientNetB0(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
```

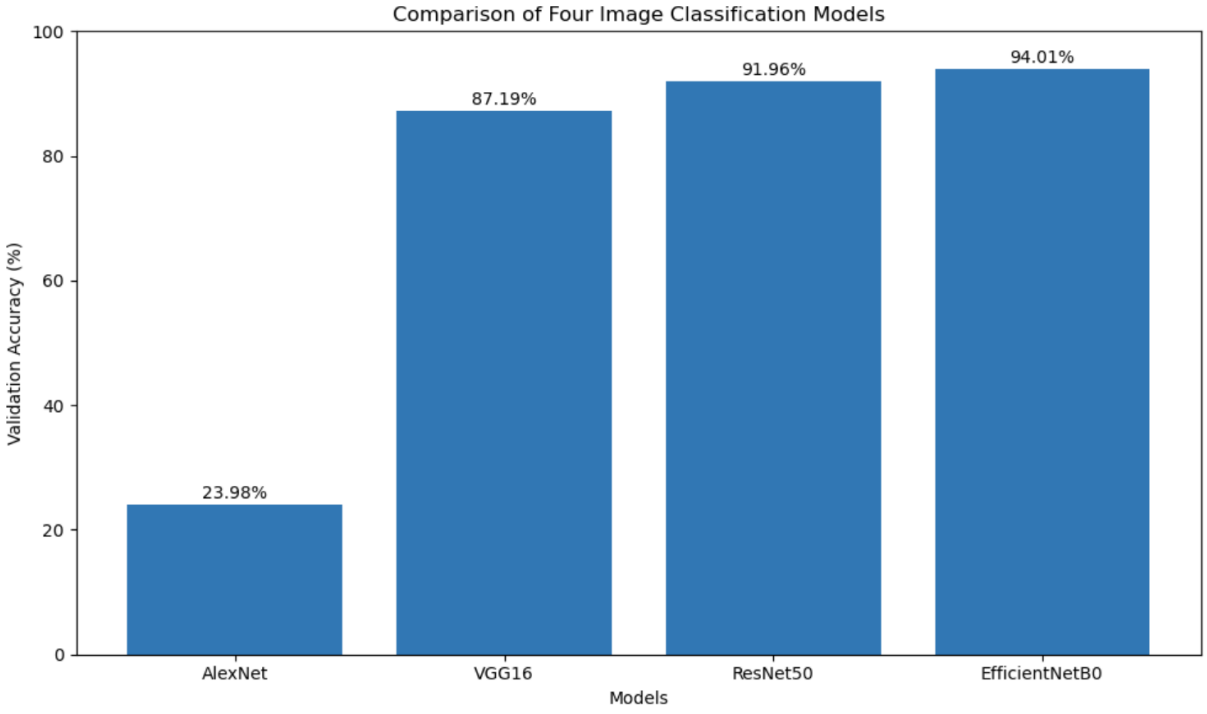
```
efficientnet_base.trainable = False
efficientnet_model = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    efficientnet_base,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(num_classes,  activation="softmax")
])
efficientnet_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("EfficientNetB0 output shape:", efficientnet_model.output_shape)
efficientnet_history = efficientnet_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS
)
efficientnet_loss, efficientnet_accuracy = efficientnet_model.evaluate(val_ds)
print("EfficientNetB0 Validation Loss:", efficientnet_loss)
print("EfficientNetB0 Validation Accuracy:", efficientnet_accuracy)
print(
    "EfficientNetB0 Validation Accuracy (%):",
    round(efficientnet_accuracy * 100, 2)
)
results = pd.DataFrame({
    "Model": [
        "AlexNet",
        "VGG16",
        "ResNet50",
        "EfficientNetB0"
    ],
    "Training Type": [
        "Trained from scratch",
        "Pre-trained",
        "Pre-trained",
        "Pre-trained"
    ],
    "Validation Loss": [
        alexnet_loss,
        vgg_loss,
        resnet_loss,
        efficientnet_loss
    ],
    "Validation Accuracy (%)": [
        alexnet_accuracy * 100,
        vgg_accuracy * 100,
        resnet_accuracy * 100,
        efficientnet_accuracy * 100
```

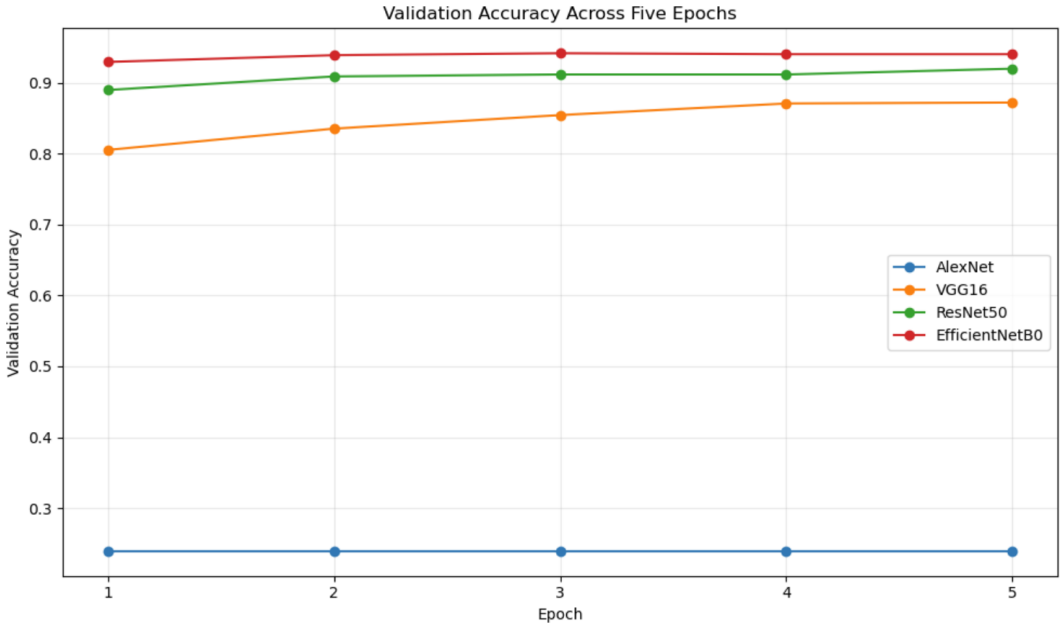
```
    ]
})
results["Validation Loss"] = results["Validation Loss"].round(4)
results["Validation Accuracy (%)"] = results["Validation Accuracy (%)"].round(2)
print(results.to_string(index=False))
model_names = results["Model"].tolist()
accuracies = results["Validation Accuracy (%)"].tolist()
plt.figure(figsize=(10, 6))
bars = plt.bar(model_names, accuracies)
plt.xlabel("Models")
plt.ylabel("Validation Accuracy (%)")
plt.title("Comparison of Four Image Classification Models")
plt.ylim(0, 100)
for bar, accuracy in zip(bars, accuracies):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height() + 1,
        f"{accuracy:.2f}%",
        ha="center"
    )
plt.tight_layout()
plt.show()

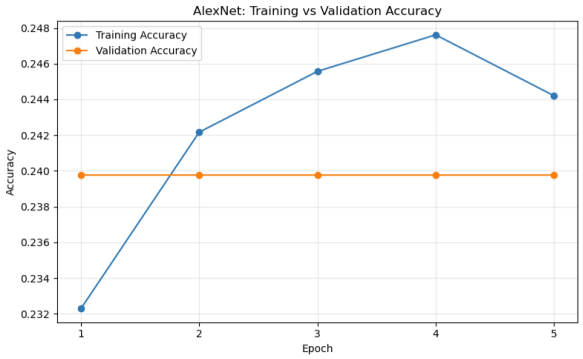
histories = {
    "AlexNet": alexnet_history,
    "VGG16": vgg_history,
    "ResNet50": resnet_history,
    "EfficientNetB0": efficientnet_history
}
plt.figure(figsize=(10, 6))
for model_name, history in histories.items():
    epochs = range(1, len(history.history["val_accuracy"]) + 1)
    plt.plot(
        epochs,
        history.history["val_accuracy"],
        marker="o",
        label=model_name
    )
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Across Five Epochs")
plt.xticks(range(1, EPOCHS + 1))
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
for model_name, history in histories.items():
    plt.figure(figsize=(8, 5))
    plt.plot(
        range(1, EPOCHS + 1),
        history.history["accuracy"],
```

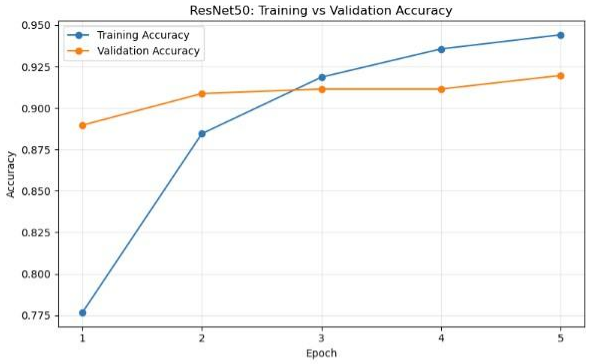
```
        marker="o",
        label="Training Accuracy"
    )
    plt.plot(
        range(1, EPOCHS + 1),
        history.history["val_accuracy"],
        marker="o",
        label="Validation Accuracy"
    )
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title(f"{model_name}: Training vs Validation Accuracy")
    plt.xticks(range(1, EPOCHS + 1))
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
best_model = results.loc[
    results["Validation Accuracy (%)"].idxmax()
]
print("Best Model:", best_model["Model"])
print("Validation Accuracy:", best_model["Validation Accuracy (%)"], "%")
print("Validation Loss:", best_model["Validation Loss"])
```

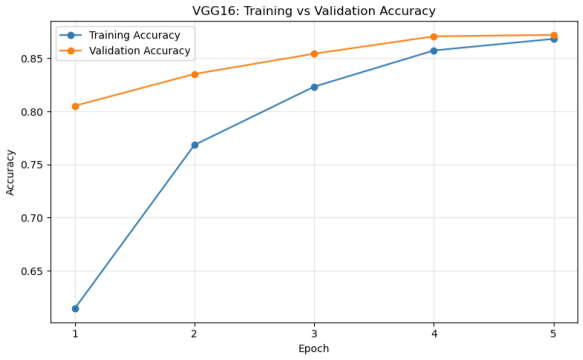
Output:

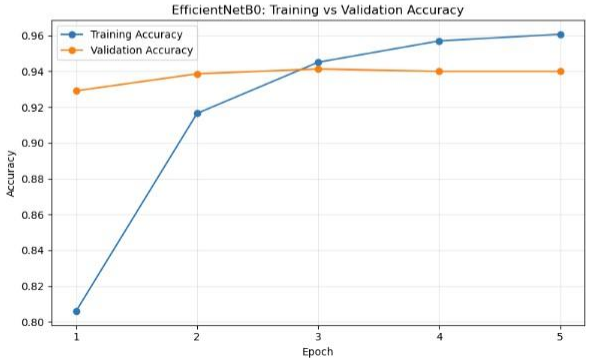



Validation Accuracy Across Five Epochs
Validation Accuracy
Epoch
AlexNet
VGG16
ResNet50
EfficientNetB0


AlexNet: Training vs Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch


ResNet50: Training vs Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch


VGG16: Training vs Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch


EfficientNetB0: Training vs Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch